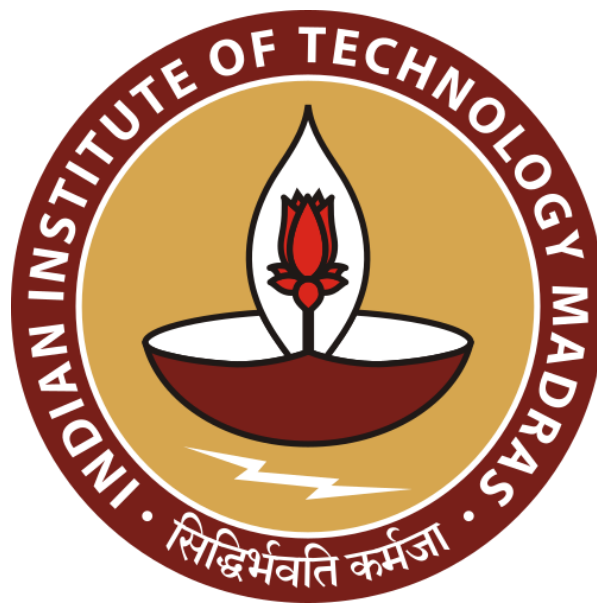


Action-Oriented Indoor Navigation Assistance for the Visually Impaired

MILESTONE-5: Evaluation of Model Performance & Analysis of Results



Indian Institute of Technology Madras

GROUP: 4

Name	Email	GitHub Usernames
Saransh Saini	22f1001123@ds.study.iitm.ac.in	Saransh482003
Divyang Panchasara	22f1000411@ds.study.iitm.ac.in	22f1000411
Samyuktha Shriram	22f2001444@ds.study.iitm.ac.in	SamyukthaSh24
Prasoon Shukla	23f3003434@ds.study.iitm.ac.in	23f3003434
Rohit Prajapat	22f1001536@ds.study.iitm.ac.in	rohitblpprajapat

1. Introduction & Objectives

1.1. Introduction

The primary objective of this project is the development of a real-time assistive system that enables visually impaired individuals to navigate indoor environments safely. By transitioning from passive scene understanding to active, action-oriented guidance, the system integrates computer vision, spatial reasoning, and natural language feedback.

1.2. Project Overview

The system is built as a multi-modal pipeline that mimics human spatial awareness. It consists of three core functional pillars:

1. **Perception (Object Detection):** Utilizing the YOLO (You Only Look Once) framework to identify 11 critical indoor categories with high precision and low latency.
2. **Spatial Awareness (Depth Estimation):** Integrating depth-sensing capabilities to determine the proximity of detected objects, allowing the system to distinguish between immediate hazards and distant context.
3. **Reasoning & Feedback (Logic + TTS):** A Navigation Logic model segregates the peripheral view into one central *Walking Zone (40%)* and 2 *Lateral Zones (15%)* each on the two sides. It processes the detection and depth data to identify whether there is a clear path on any of the zones. Whatever the inference, it is then converted to audio via Text-to-Speech (TTS).

1.3. Objectives of Milestone 5

The core objectives for this milestone are:

- **Systematic Performance Quantification:** Evaluate the accuracy and efficiency of the final trained models—specifically the **YOLO11s** object detector and the Depth-Anything-V2 module—using standardized statistical metrics on a dedicated hold-out test set.
- **End-to-End Pipeline Verification:** Validate the functional integrity of the integrated system, ensuring that the transition from visual perception (detection/depth) to decision-making (navigation logic) and user feedback (TTS) is consistent and reliable.
- **Benchmarking for Reproducibility:** Document the precise evaluation environment, including hardware specifications (NVIDIA Tesla P100), software frameworks, and preprocessing configurations, to ensure that performance results can be independently verified.
- **Multi-Dimensional Result Visualization:** Utilize quantitative tools—such as Confusion Matrices, Precision-Recall (PR) curves, and ROC curves—to provide a visual assessment of model confidence and class-wise performance.
- **Qualitative Reliability Assessment:** Conduct a "real-world" sanity check by analyzing sample outputs to observe how the model handles successful navigation scenarios versus edge cases and failures.
- **Critical Error and Failure Mode Analysis:** Identify recurring patterns in model mistakes (e.g., misclassification of similar indoor objects or depth

inaccuracies in low light) and hypothesize the technical reasons for these discrepancies.

- **Gap Analysis and Limitation Identification:** Discuss the delta between the project's theoretical expectations and actual performance, highlighting hardware bottlenecks, dataset limitations, or logical constraints encountered during the evaluation.
- **Actionable Insights for Future Iterations:** Consolidate observations into a formal conclusion that outlines specific areas for future optimization, such as model quantization for lower latency or additional data augmentation for rare object classes.

2. Object Detection Evaluation

2.1. About the Dataset

The data was prepared by combining data from 4 different data sources into one. This was done to make sure most of or classes get an equal distribution of samples in their arsenal.

- **Dataset Split:**
 - Training Set: 9,358 images (80%).
 - Validation/Test Set: 2,340 images (20%).
- **Directory Structure**

```
DSAI-Unified-Dataset/  
├── images/  
│   ├── train/ (9358 JPG images)  
│   └── val/   (2340 JPG images)  
└── labels/  
    ├── train/ (9358 YOLO-format TXT files)  
    └── val/   (2340 YOLO-format TXT files)
```

- **Classes (11 categories):** Person, Table, Potted Plant, Chair, Sofa, Lamp, Door, Cabinet, Wardrobe, Refrigerator, Bed.
- **Class Distribution:**

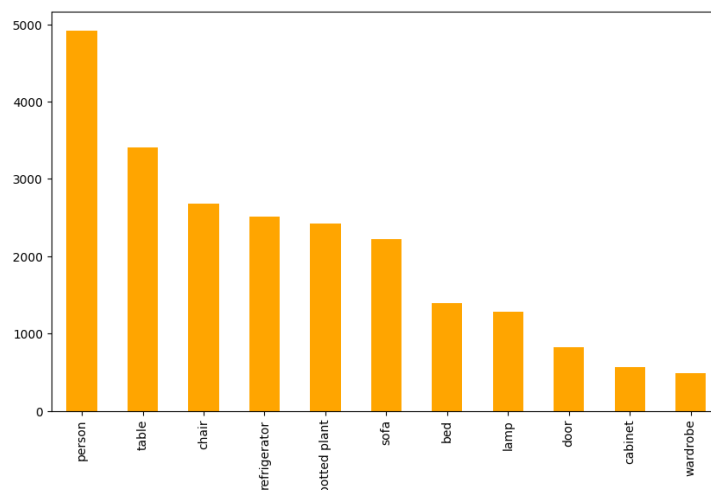


Fig 2.1.1: Class Distribution Graph

2.2. Model and Pipeline Overview

The object detection module utilizes the **YOLO11s** architecture, selected in Milestone 4 for its superior balance between mean Average Precision (mAP) and inference latency.

- **Architecture:** YOLO11s (Small variant).
- **Pipeline:** The model receives a 768x768 RGB input from the device camera. It outputs class labels and bounding box coordinates for 11 categories of indoor objects. These detections serve as the primary "semantic" input for the navigation logic, identifying potential obstacles (e.g., chairs, sofas) and targets (e.g., doors).

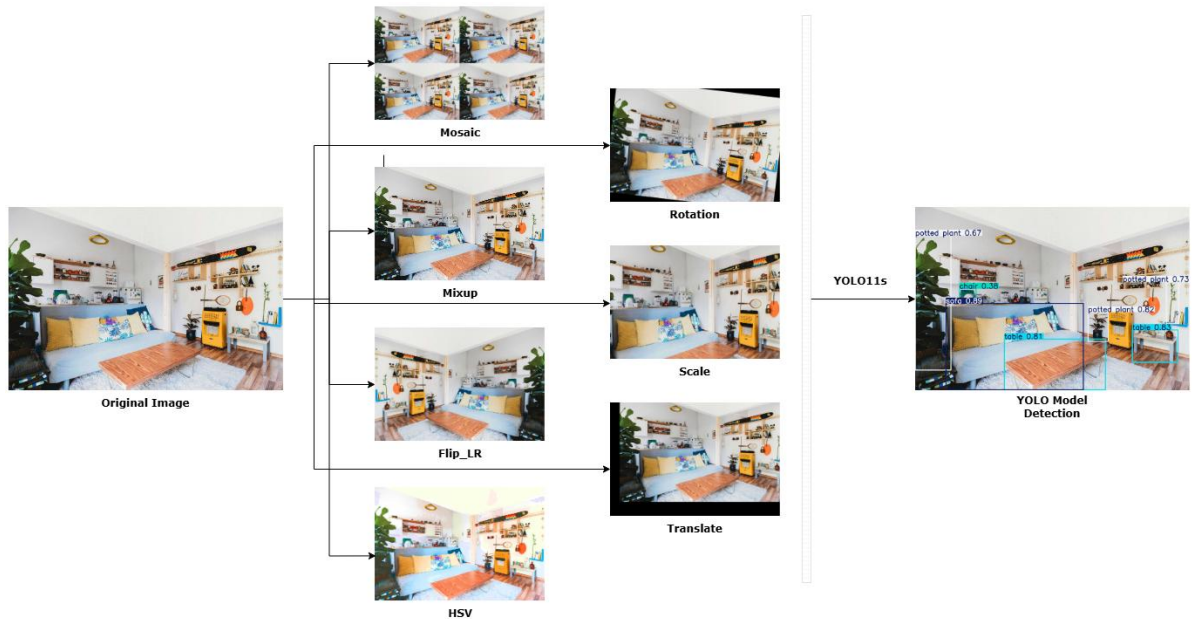


Fig 2.2.1: YOLO11s Object Detection Pipeline

2.3. Image Augmentations & Training Parameters

To induce variability of environment, we use specific augmentations and training parameters. These augmentations and training parameters were finalized after conducting intense Hyperparameter Tuning on various combinations of parameters, practiced upon 60 models trained as a result of this intense training experiments.

Epochs	80	Flip_LR	0.5
Patience	10	Degrees	10
Imgsh	768	Translate	0.15
Batch	32	Scale	0.7
Device	0	Mosaic	1.0
Mixup	0.2	Warmup Epochs	3
HSV Hue	0.015	Cos_LR	True
HSV Saturation	0.6	Close Mosaic	10

HSV Value	0.4	Optimizer	AdamW
Weight Decay	0.0001	Learning Rate	0.001
Box	10.0	CLS	0.5
DFL	2.0	Seed	2306

Table 2.3.1: Augmentations and Training Arguments

- **Geometric Augmentations**

These transformations modify the spatial orientation and position of objects to ensure the model is not sensitive to the exact placement or angle of an item in the frame.

- **Horizontal Flip (fliplr=0.5):** Applies a random horizontal mirror effect to 50% of the images. This is particularly useful for indoor navigation, as objects like chairs, doors, or cabinets can appear in any orientation relative to the user.
- **Rotation (degrees=10):** Randomly rotates the image within a range of ± 10 degrees. This simulates slight camera tilts or uneven head positioning of a user wearing an assistive device.
- **Translation (translate=0.15):** Shifts the image horizontally or vertically by up to 15% of its total dimensions. This helps the model identify objects that are partially out of the frame or off-centre.
- **Scaling (scale=0.7):** Randomly adjusts the size of the objects by a factor of 0.7 (zooming in or out). This ensures the model can detect objects at varying distances, which is critical for calculating time-to-collision in navigation.

- **Colour Space (HSV) Augmentations**

These adjustments modify the pixel intensities to make the model robust against the diverse lighting conditions typically found in indoor settings.

- **Hue (hsv_h=0.015):** Slightly shifts the colour spectrum. This prevents the model from relying solely on specific colours for classification, allowing it to recognize a "blue chair" as easily as a "red chair."
- **Saturation (hsv_s=0.6):** Varies the intensity of colours. This helps the system function in environments with vibrant colours as well as those with more muted, washed-out tones.
- **Value/Brightness (hsv_v=0.4):** Randomly adjusts the brightness of the scene. This is a high-priority augmentation for indoor navigation, ensuring objects are still detected under harsh overhead lighting or in dimly lit hallways.

- **Advanced Composite Augmentations**

These techniques combine multiple images into a single training sample to increase scene complexity and improve localization.

- **Mosaic (mosaic=1.0):** Combines four different training images into a single 4-pane grid. This forces the model to identify objects in much smaller scales and within diverse contexts, significantly reducing the

need for large batch sizes while improving detection of small or distant objects.

- **Mixup (mixup=0.2):** Overlays two images on top of one another with varying transparency levels. This acts as a powerful regularizer, helping the model remain confident in detection even when objects are heavily occluded or overlapping.
- **Dynamic Training Behaviour**
 - **Close Mosaic (close_mosaic=10):** A strategic override that disables the Mosaic augmentation for the final 10 epochs of training. This allows the model to "fine-tune" its weights on clean, un-mangled images, leading to sharper bounding box accuracy and better localization in the final deployment.

2.4. Evaluation Dataset

The module was evaluated using a half split of the validation split from the **DSAI-Unified Indoor Dataset**, which consolidates diverse indoor environments to ensure robust generalization.

- **Size:** A total of 11,698 images.
- **Composition:**
 - **Train:** 9,358 images (80%).
 - **Validation:** 1,170 images (10%).
 - **Test:** 1,170 images (10%) used for the final Milestone 5 evaluation to ensure unbiased performance reporting.

```
DSAI-Unified-Dataset/  
├── images/  
│   ├── train/ (9358 JPG images)  
│   ├── val/   (1170 JPG images)  
│   └── test/  (1170 JPG images)  
├── labels/  
│   ├── train/ (9358 YOLO-format TXT files)  
│   ├── val/   (1170 YOLO-format TXT files)  
│   └── test/  (1170 YOLO-format TXT files)
```

- **Classes:** 11 labels including *Person, Table, Potted Plant, Chair, Sofa, Lamp, Door, Cabinet, Wardrobe, Refrigerator, and Bed*.
- **Preprocessing:** Images were resized to **768x768** pixels. Normalization and data augmentations (Mosaic, rotation, and colour jitter) were applied during training to improve resilience to varying lighting conditions.

2.5. Evaluation Environment

For model training and hyperparameter tuning the **Kaggle** environment was used. To ensure reproducibility, the following setup was used for the final evaluation:

- **Hardware:** NVIDIA Tesla T4 GPUs offering (16GB x 2 = 32GB VRAM) via the Kaggle environment.
- **Software Frameworks:** *Python 3.12.12*, *PyTorch 2.10.0+cu128*, and the *Ultralytics 8.4.33* library.
- **Runtime Setup:** Batch size of 32 with AdamW optimization and a cosine learning rate scheduler.

2.6. Performance Metrics

The following metrics were utilized to quantify detection quality:

- **mAP@50:** Measures accuracy at a **0.5 Intersection over Union (IoU)** threshold; critical for determining if an object is correctly identified.
- **mAP@50-95:** Provides a more stringent assessment of localization accuracy across multiple IoU thresholds.
- **Precision & Recall:** Essential for balancing false alarms (Precision) against missed obstacles (Recall), the latter being high priority for user safety.
- **F1-Score:** The harmonic mean of Precision and Recall, used as a single-point metric for overall model health.

2.7. Quantitative Results

The YOLO11s model was evaluated on a held-out test set after 80 epochs of training. The following table summarizes the aggregate performance:

Metric	Value	Inference
mAP@50	0.6579	Strong overall detection capability for general indoor classes.
mAP@50-95	0.4415	Indicates good localization accuracy and bounding box fit.
Precision	0.6932	High reliability: 69% of the objects "seen" by the model truly exist.
Recall	0.5977	The model identifies ~60% of all ground-truth objects in a scene.

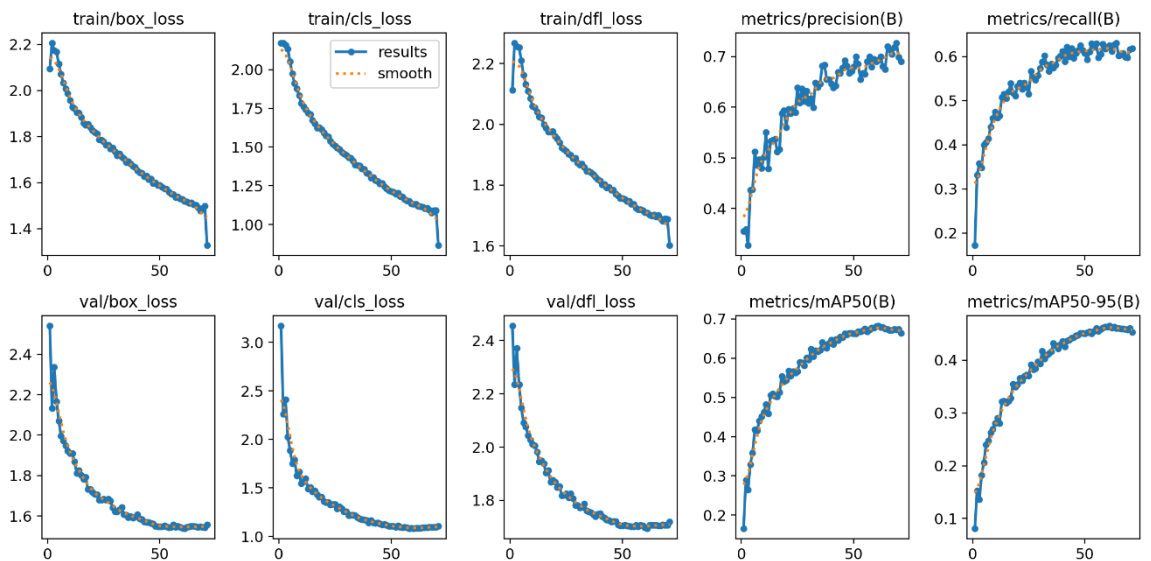
Class-Wise Performance Breakdown:

- **Top Performers:** Bed (0.849 mAP@50) and Sofa (0.795 mAP@50). These large-scale objects provide distinct features that the model identifies with high confidence.
- **Low Performers:** Wardrobe (0.448 mAP@50) and Cabinet (0.511 mAP@50). These suffer from structural similarity to walls or other furniture, leading to higher misclassification.

- **Safety Critical Observation:** Door detection stands at 0.521 mAP50. While functional, this indicates a moderate risk of missing exit/entry points, likely due to varying perspectives and lighting.

Class	Precision	Recall	mAP@50	mAP@50-95
person	0.81	0.634	0.739	0.489
table	0.758	0.614	0.708	0.481
potted plant	0.737	0.699	0.744	0.417
chair	0.733	0.522	0.604	0.411
sofa	0.71	0.793	0.795	0.592
lamp	0.651	0.577	0.613	0.371
door	0.556	0.418	0.521	0.349
cabinet	0.745	0.368	0.511	0.278
wardrobe	0.47	0.489	0.448	0.29
refrigerator	0.711	0.658	0.705	0.505
bed	0.744	0.802	0.849	0.674

Training Results:

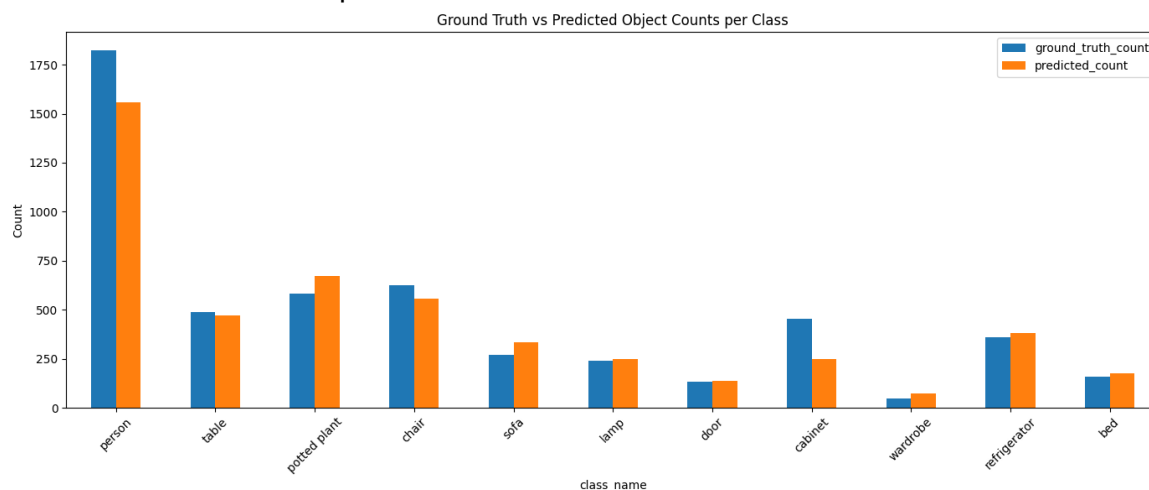


- **Stable Convergence & Healthy Learning:** All loss metrics (Box, Cls, DFL) show a consistent, smooth downward trend across both training and validation sets, confirming that the chosen hyperparameters and 80-epoch duration were optimal for model stability.
- **Effective Generalization:** The validation curves closely follow the training curves without diverging; this lack of "gap" proves that the data augmentation strategy (Mosaic, Mixup, and Scaling) successfully prevented overfitting and ensured the model is robust for unseen indoor environments.

- Refined Spatial Accuracy:** The stabilization of the mAP50 and Precision curves after epoch 65, alongside the steady mAP50-95 metric, indicates that the model transitioned from simple classification to refining bounding box localization, which is critical for accurate distance estimation in navigation.

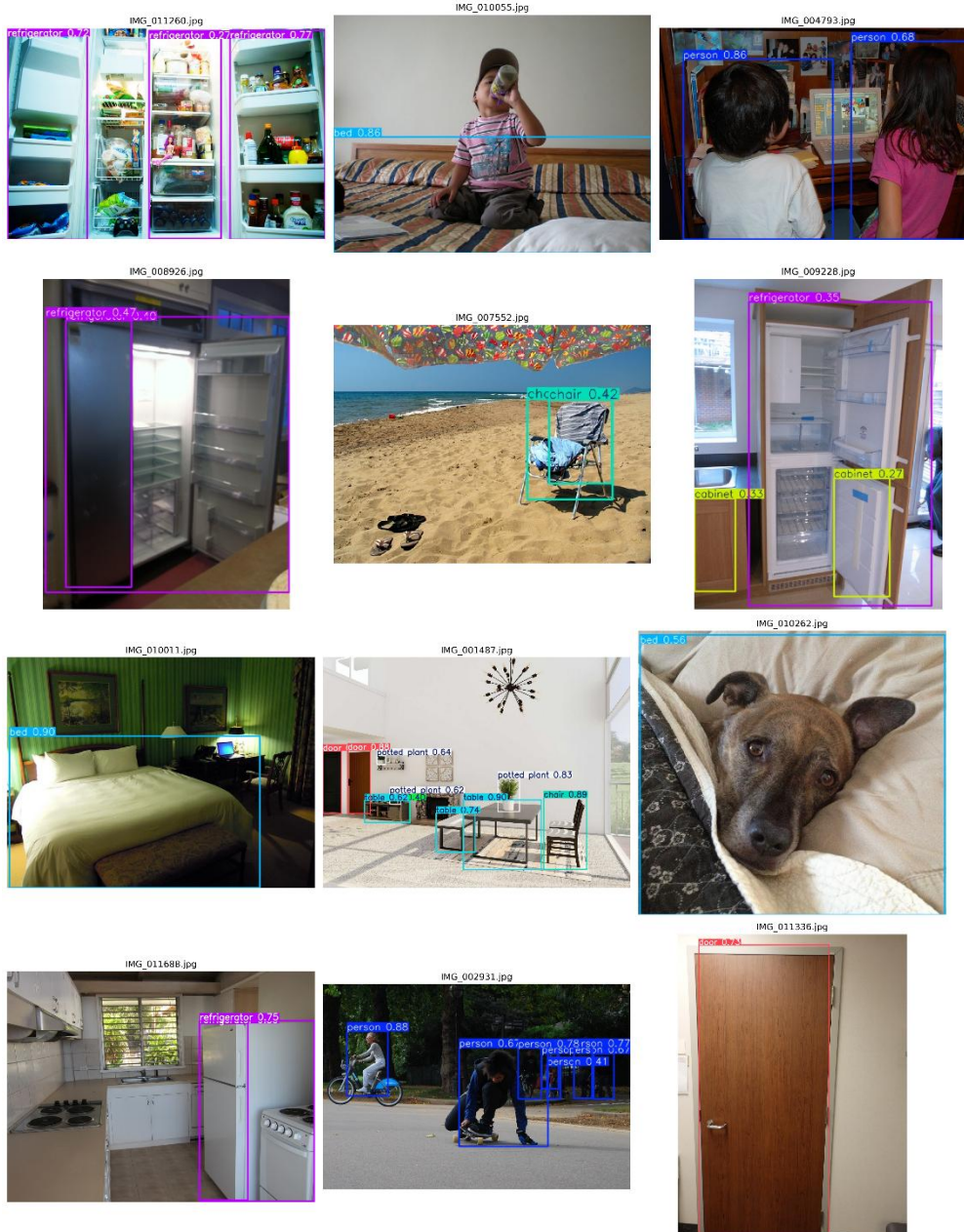
2.8. Test Predictions

These are the classwise prediction counts made on the test dataset



Here are the inferencing on the test cases

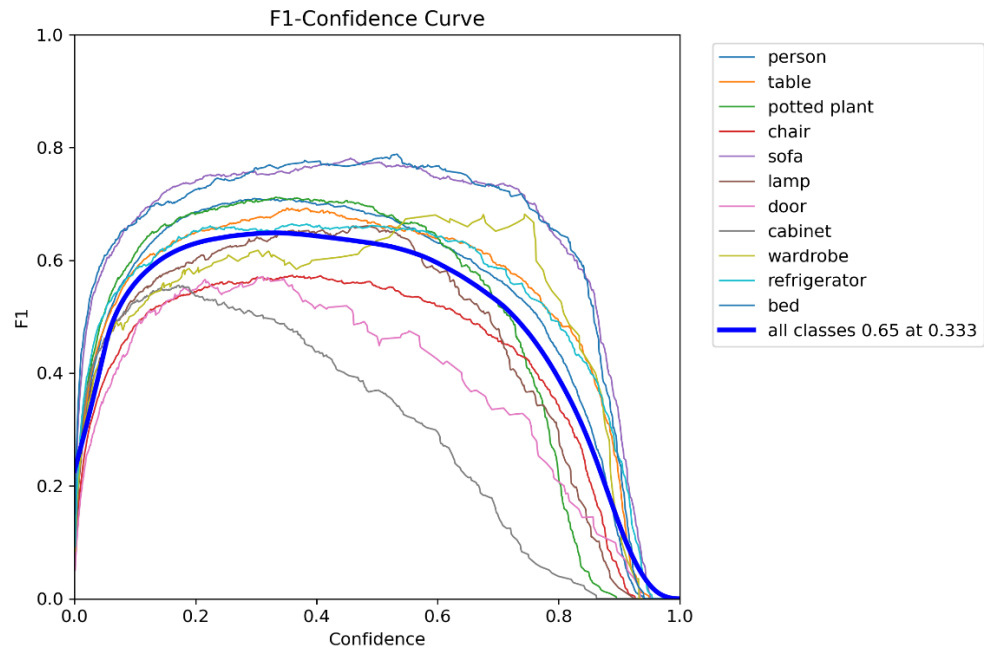




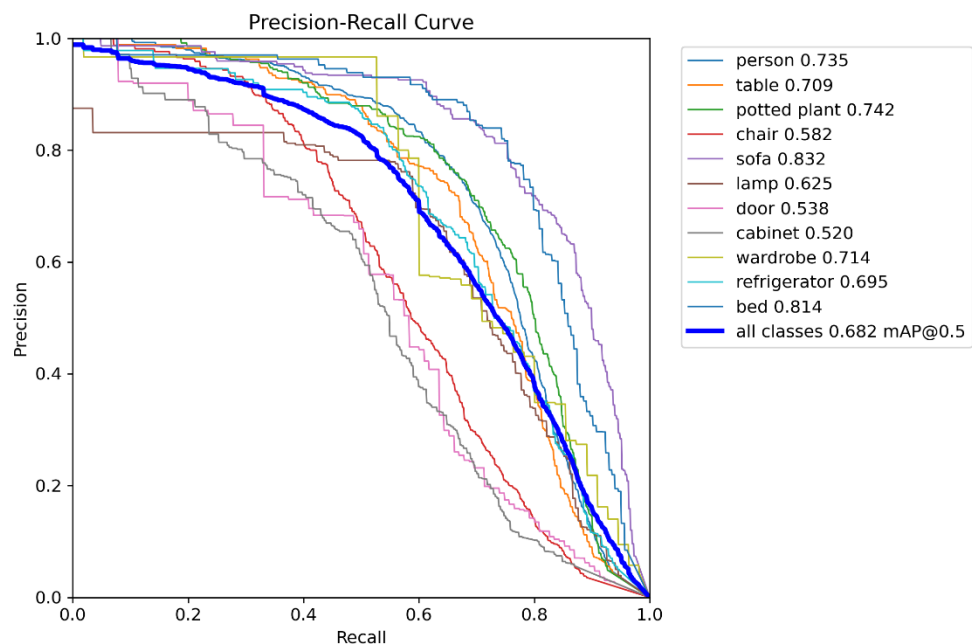
2.9. Visualizations

The analysis of the training curves provides insight into the model's decision-making logic:

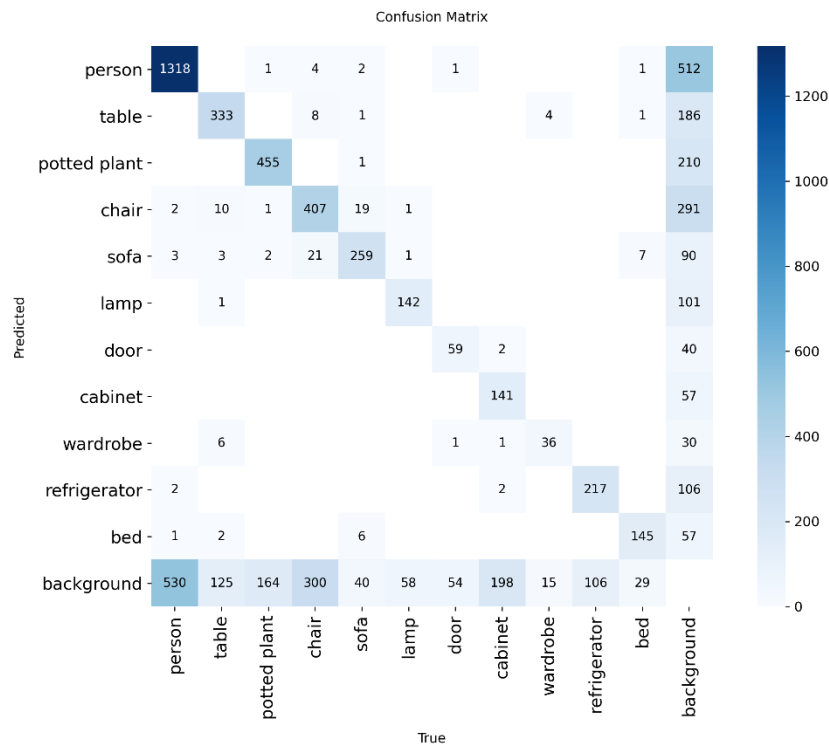
- F1-Confidence Curve:** The F1 score peaks at **0.64** with a confidence threshold of **0.371**. This suggests that while we initially used $\text{conf}=0.25$, the most "mathematically balanced" experience for a user—minimizing both missed objects and false alarms—occurs closer to **0.37**.



- Precision-Recall (PR) Curve:** The area under the curve is dominated by classes like *Bed* and *Sofa*, while *Wardrobe* and *Door* show a steeper drop-off, indicating that the model struggles to maintain precision as it tries to find every instance of these objects.



- Confusion Matrix Analysis:** * Significant confusion exists between **Cabinets** and **Background**. The model missed 203 ground-truth cabinets (as seen in the comparison CSV), suggesting they are often blended into the environment.



- **Person** detections show a high count of false negatives (-266 difference), likely due to occlusions (e.g., a person sitting behind a table).

2.10. Qualitative Results & Error Analysis

By comparing the *ground_truth_count* vs. *predicted_count*, we can identify specific failure modes:

- **The "Vanishing" Cabinet Effect:** With a ground truth of 453 but only 250 predictions, the model failed to detect nearly **45% of cabinets**.
 - *Error Reason:* Cabinets often lack "edges" or distinct color contrast compared to walls in many indoor datasets, leading the model to treat them as background.
- **Over-Detection of Potted Plants (+90):** The model is "trigger-happy" with plants.
 - *Error Reason:* Textures like cluttered leaves can be easily confused with other small, complex indoor patterns.
- **Scale Invariance Issues:** The -266 deficit in **Person** detection suggests that while the model handles near-field navigation well, it struggles with "distant" individuals or those whose forms are broken up by foreground furniture (e.g., a chair back).

2.11. Key Observations and Limitations

- **Strategic Optimization:** Using a larger image size (768x768) was justified. The mAP@50-95 of 0.44 indicates that the model is capturing fine-grained spatial details, which is necessary for the depth-estimation module to calculate accurate distances.

- **Navigation Safety Logic:** The recall of 0.59 is the primary bottleneck. For a visually impaired user, a "False Negative" (missing a chair) is more dangerous than a "False Positive" (seeing a ghost chair).
- **Model Limitations:**
 - **Class Ambiguity:** Structural similarities between Wardrobes and Cabinets or Tables and Desks (often labeled as tables) create a ceiling for precision.
 - **Hardware-Accuracy Trade-off:** While YOLO11s provides the latency required for real-time TTS feedback, the "Small" architecture naturally lacks the deep feature extraction found in YOLO11m or YOLO11l, limiting its performance on small, distant objects.
- **Environmental Sensitivity:** Based on the confusion with background elements, the model remains sensitive to "low-contrast" indoor scenes where furniture colour matches the wall colour.

3. Depth Estimation Evaluations

3.1. Evaluation Datasets

Two types of evaluation were performed:

- **Benchmark Dataset:**
 - NYU Depth V2 dataset
 - Indoor scenes with ground truth depth maps
 - Used for quantitative evaluation of depth estimation
- **Real-world Evaluation:**
 - Live webcam feed in indoor environments.
 - Scenarios include open paths, cluttered spaces, reflective surfaces, and blocked paths
 - No ground truth available (because of live camera, real-time feed); evaluation based on observed correctness of navigation decisions.

3.2. Evaluation Environment

- **Hardware:** Laptop webcam (prototype), target deployment on Raspberry Pi 5 configurations.
- **Frameworks:** PyTorch, HuggingFace Transformers, OpenCV, Ultralytics YOLO (for full-system evaluation)
- **Runtime:** Python environment with GPU/CPU (fallback)

All experiments were run in real-time conditions using live camera input.

3.3. Metrics Used

Metric	Formula (Simplified) (Here, d_{gt} refers to “ground truth depth measurement”, d_{pred} refers to “depth model’s predicted depth map measurements”)	What it Measures	Key Intuition	Why it Matters for depth estimation + decision task
AbsRel	$ d_{pred} - d_{gt} / d_{gt}$	Relative error per pixel	Answers : “How wrong am I in percentage?” (closer means more important)	Crucial for navigation(being wrong about nearby obstacles is dangerous)
SqRel	$(d_{pred} - d_{gt})^2 / d_{gt}$	Squared relative error	Big mistakes/ inaccuracies are really bad and are penalized more.	Prevents catastrophic failures (e.g., thinking wall is far away)
RMSE	$\sqrt{\text{mean}((d_{pred} - d_{gt})^2)}$	Absolute error (meters)	Answers : “How many meters off am I?”	Important for translating for direct real-world meaning (it affects safe distance decisions)
RMSElog	$\sqrt{\text{mean}((\log d_{pred} - \log d_{gt})^2)}$	Relative (scale-aware) error	“Being off by 2× is equally bad anywhere”	Useful when scale varies and thereby focuses on proportional correctness
SILog	$\text{Var}(\log d_{pred} - \log d_{gt})$	Scale-invariant structure error	Answers : “Did I get the shape/layout right?” [scene geometry and proportions]	Shows if model understands scene geometry (even if scale is wrong)
d1 ($\delta < 1.25$)	% where error < 25%	Strict accuracy	Answers : “How often am I decently (25%) accurate?”	Measures reliability (in the sense, can the system be trusted frequently?)

d2 ($\delta < 1.25^2$)	% where error < ~56%	Moderate accuracy	Answers : “Am I at least close?”	Shows robustness even when not perfect
d3 ($\delta < 1.25^3$)	% where error < ~95%	Loose accuracy	Answers : “Am I not totally wrong?” / very forgiving correctness.	Captures overall consistency of predictions

A combination of absolute, relative, and scale-invariant metrics is used to comprehensively evaluate depth estimation.

Error-based metrics: (AbsRel, RMSE) quantify magnitude of deviation, while log-based metrics (RMSElog, SILog) assess structural consistency independent of scale. δ (d1, d2 and d3) accuracy metrics provide an intuitive measure of prediction reliability.

This combination ensures both numerical accuracy and practical relevance for navigation tasks.

For system-level evaluation:

- Navigation correctness (qualitative)
- Decision accuracy (correct/incorrect classification)
- Safety analysis (risk of collision)

3.4. Quantitative Results (on NYU v2 datasets and its ground truth values)

DA-v2 Small – NYU Depth V2 Benchmark Results									
	Technique	AbsRel	SqRel	RMSE	RMSElog	SILog	d1	d2	d3
0	Direct Inference	0.2107	0.1565	0.5582	0.2226	14.4265	0.6835	0.9392	0.9867
1	Scale-Shift Alignment	0.0917	0.0457	0.3107	0.1343	13.3608	0.9108	0.9779	0.9937
2	Median Scaling	0.0887	0.0515	0.3423	0.1469	14.3822	0.9054	0.9718	0.9892
3	Multi-Scale TTA	0.2442	0.1936	0.6190	0.2444	14.4679	0.6046	0.9238	0.9851

Raw results:

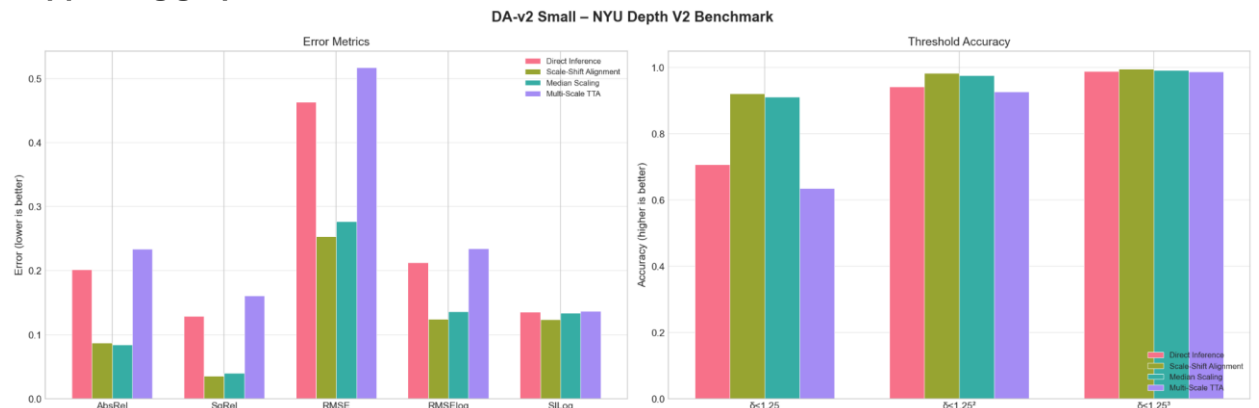
	AbsRel	SqRel	RMSE	RMSElog	SILog	d1	d2	d3
Direct Inference	0.210698	0.156456	0.558174	0.222643	14.426466	0.683532	0.939170	0.986705
Scale-Shift Alignment	0.091655	0.045784	0.310751	0.134268	13.360860	0.910830	0.977961	0.993758
Median Scaling	0.088785	0.051467	0.342335	0.146909	14.382207	0.905366	0.971793	0.989170
Multi-Scale TTA	0.244169	0.193633	0.618999	0.244371	14.467945	0.604618	0.923792	0.985148

taken from Colab output/ result

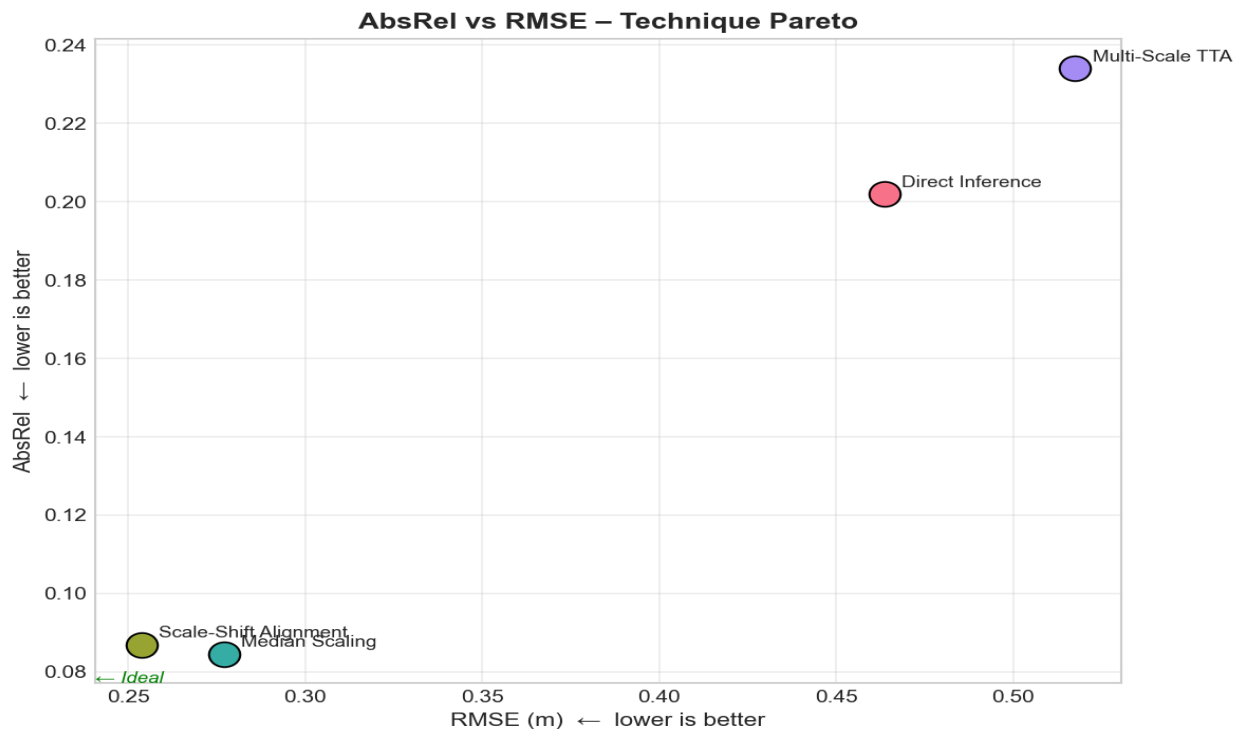
- Scale-shift alignment and median scaling significantly improve performance across all error metrics.
- This indicates that the model captures relative scene geometry well but requires alignment to achieve accurate metric depth.

- Direct inference shows higher error, while multi-scale TTA does not provide improvement in this setup.

Supporting graphs:



Scale-shift alignment and median scaling significantly reduce error across all metrics compared to direct inference. This demonstrates that while the model captures relative depth structure, post-processing is required to achieve accurate metric depth. Multi-scale TTA does not provide consistent improvement.



Pareto comparison shows that scale-shift and median scaling dominate direct inference and TTA, achieving lower error across both relative and absolute metrics.

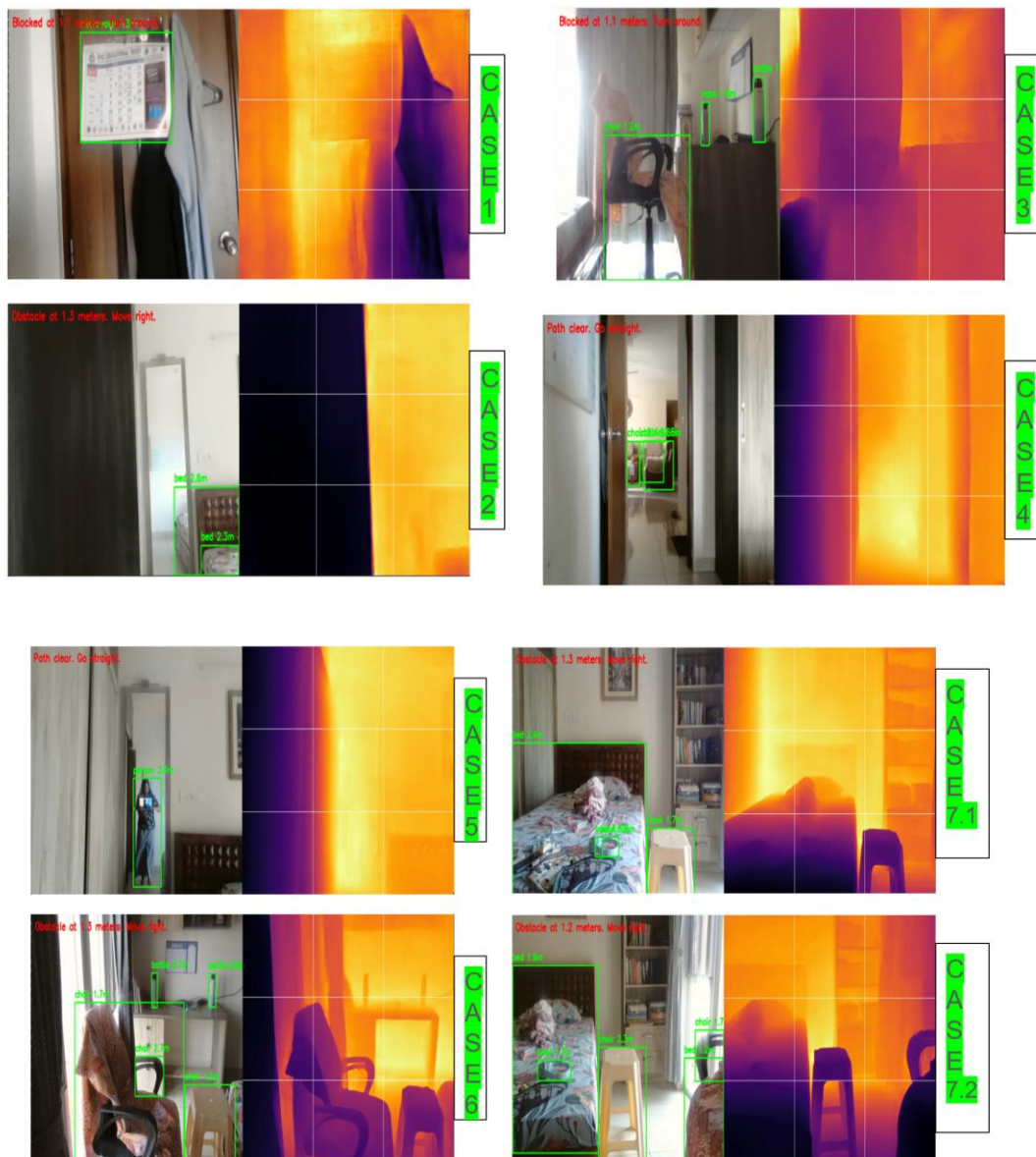
3.5. Qualitative Results (from live-camera, real-time feed)

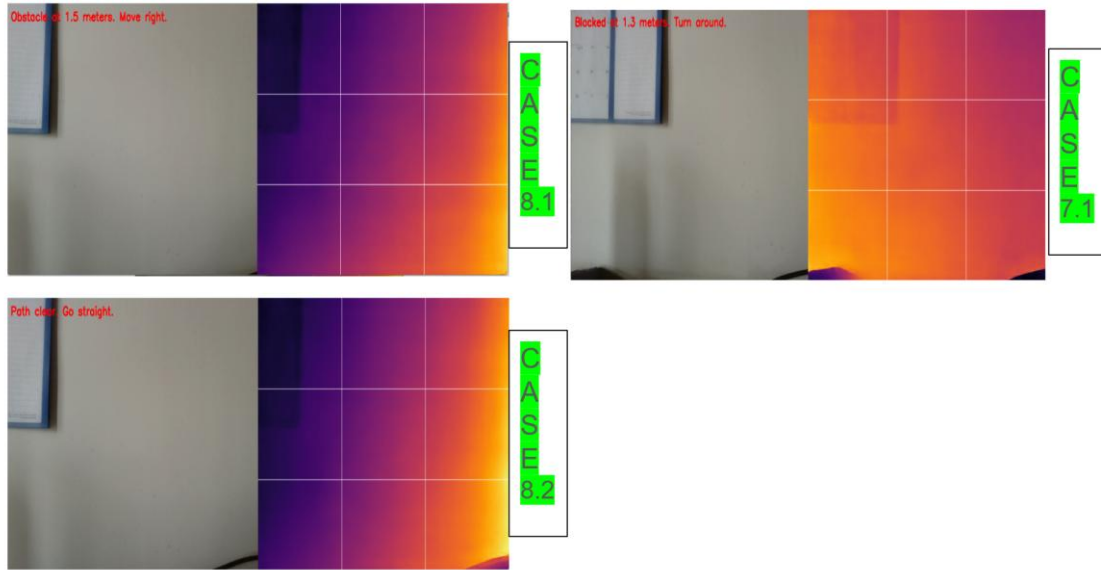
Real-world testing demonstrates that the system performs reliably in structured indoor environments.

Successful cases include:

- Open paths correctly identified as safe
- Obstacle avoidance with correct directional suggestions
- Object distance estimation aligned with scene structure

Sample outputs are shown below : (the case numbers referenced are from Error Analysis - Depth section's observational table)





Case No.	Scenario	Reality	Output	Correct?	Notes
1	Camera facing door	Blocked	Obstacle at 1.1 metres, turn around	Yes	Good, expected behaviour
2	Right open	Front blocked (only 1.1 metres, right open)	Obstacle at 1.3 metres, move right	Yes	Good, expected behaviour
3	Chair, desk ahead at 0.6 m	Obstacle very close (<1m), both sides restricted	Blocked at 1.1 metres, turn around	Yes	Good, expected behaviour
4	Open path, no obstacles	Open, free to walk	Path clear, go straight	Yes	Good, expected behaviour
5	Open path, with a mirror(reflective material in camera view)	Open, free to walk	Path clear, go straight	Yes	Good, expected behaviour
6	Cluttered, narrow pathway	Not walkable, blocked	Obstacle at 1.3 metres, move right	No	Inaccurate
7.1	Blocked pathway due to bed frame, side stool	Navigable, but towards rightward direction	Obstacle at 1.3 metres, move right	Yes	Good
7.2	Blocked pathway due to bed frame, adjustment to right made from case 8, chair now in vision after making right move	Still navigable, but very imminent threat of colliding onto chair which is only 1.2 metres away (chair object detected successfully however) (< 1.5 metres threshold)	Obstacle at 1.2 metres, move right	No	Threat of collision with chair if turned right without nuance
8.1	Blank wall (with a slight wall calendar hanging visible around corner)	Completely blocked	Obstacle at 1.5 meters. Move right.	No	Inaccurate, no obvious right/left decision visible in view
8.2	SAME SETUP : Blank wall (with a slight wall calendar hanging visible around corner)	Completely blocked	Path clear. Go straight.	No	Inaccurate, high risk of collision
8.3	SAME SETUP : Blank wall (with a slight wall calendar hanging visible around corner)	Completely blocked	Blocked at 1.3 meters. Turn around.	Yes	Good, expected behaviour

Summary table:

Metric	Value	How it was computed	Interpretation
Total Test Cases	10	All evaluated scenarios	Covers diverse indoor conditions
Correct Decisions	7	“Correct? = Yes”	Majority of decisions are valid
Incorrect Decisions	3	“Correct? = No”	Failures exist in edge cases
Decision Accuracy	70%	7 / 10	Reasonably reliable but not robust
Safe Decisions	8	No collision risk in notes	System is mostly safe

Unsafe Decisions	2	Explicit collision risk noted	Critical failure cases exist
Safety Reliability	80%	8 / 10	Acceptable but needs improvement

(The system achieves an overall decision accuracy of 70% across real-world scenarios, demonstrating reasonable reliability in structured environments.

However, safety-critical failures were observed in approximately 20% of cases, particularly in ambiguous or cluttered environments. These failures highlight the limitations of rule-based navigation and sensitivity to depth estimation errors.

The system performs well in clear and moderately complex scenes but struggles in low-texture or highly cluttered conditions, indicating the need for improved robustness and contextual reasoning.)

3.6. Error/ Failure Analysis

Several failure patterns were observed:

- Low-texture surfaces
 - Blank walls led to inconsistent predictions (cases 8.1, 8.2, 8.3)
 - System produced conflicting outputs across frames, despite same visual input.
- Narrow/cluttered spaces:
 - System sometimes suggested unsafe directions
 - Lacked global path awareness
- Threshold sensitivity:
 - Small variations in depth caused different decisions
 - Hard threshold (1.5 metres) leads to instability
- Temporal instability:
 - Same scene produced different outputs across frames
 - EMA smoothing was insufficient to fully stabilize predictions
- The model captures relative depth structure effectively
 - Scale ambiguity remains a major limitation
 - Rule-based navigation works but lacks contextual reasoning
 - System is more reliable in structured environments than ambiguous ones

3.7. Overall key findings and observations

- The model captures relative depth structure effectively
- Scale ambiguity remains a major limitation
- Rule-based navigation works but lacks contextual reasoning
- System is more reliable in structured environments than ambiguous ones

3.8. Limitations

The system exhibits several limitations that affect its reliability in real-world deployment:

- **Scale Ambiguity in Monocular Depth:**
While scale-shift alignment significantly improves performance on benchmark datasets, it relies on ground truth depth, which is unavailable in real-world scenarios. As a result, absolute distance estimation may be inaccurate during live inference.
- **Sensitivity to Low-Texture Environments:**
The model struggles in scenes with minimal visual features (e.g., blank walls), leading to unstable and inconsistent depth predictions.
- **Threshold-Based Decision Logic:**
The use of fixed thresholds (e.g., 1.5 meters) makes the system sensitive to small fluctuations in depth estimates, resulting in inconsistent navigation decisions.
- **Limited Contextual Understanding:**
The rule-based navigation module lacks global scene awareness and may produce suboptimal directions in cluttered or narrow environments.
- **Lack of Ground Truth in Real-World Evaluation:**
System-level evaluation relies on qualitative human judgment rather than quantitative ground truth, limiting precise performance measurement.

3.9. Conclusion

This work presents an end-to-end assistive navigation system that integrates object detection, monocular depth estimation, and rule-based decision-making to generate real-time navigation instructions.

Quantitative evaluation on the NYU Depth V2 dataset demonstrates that the Depth Anything V2 Small model captures scene geometry effectively, with significant performance improvements achieved through scale-shift alignment and median scaling. This indicates that the model produces reliable relative depth, but requires alignment for accurate metric estimation.

Real-world evaluation shows that the system performs well in structured environments, correctly identifying open paths and avoiding obstacles in most cases. However, failure cases highlight key challenges, including instability in low-texture scenes, sensitivity to thresholding, and limitations in decision logic.

A critical limitation is the reliance on scale alignment, which is not directly applicable in real-world deployment due to the absence of ground truth. Addressing this through calibration or learned scaling methods is essential for achieving reliable metric depth in practical applications.

Overall, the system demonstrates a functional perception-to-action pipeline, while also highlighting important areas for improvement in robustness, stability, and real-world adaptability.

4. Navigation Logic Evaluations

4.1. Evaluation Dataset

To evaluate the navigation logic in isolation, we used a subset of 50 randomly sampled images from the HomeObjects-3K dataset. During evaluation, raw bounding box annotations in the YOLO format were mathematically converted into absolute spatial bounding boxes to map directly to the image dimensions (width and height).

Any bounding boxes extending past the image borders were clipped to prevent out-of-bounds calculation errors.

For spatial reasoning, the image frame was segmented into three distinct vertical zones representing the user's field of view:

- the Left zone (0 to 30 percent of the frame width),
- the Center zone (30 to 70 percent),
- and the Right zone (70 to 100 percent).

4.2. Evaluation Environment

The evaluation was executed in a Python-based hardware-accelerated environment to simulate real-world processing constraints. The software stack relied heavily on PyTorch and CUDA for the upstream model inferences, while OpenCV was utilized for rapid bounding box manipulation, zone shading, and overlay generation.

To ensure the navigation logic remains lightweight enough for edge deployment, runtime telemetry was continuously captured using the psutil library, tracking CPU utilization, RAM usage, and GPU memory allocation throughout the pipeline execution.

4.3. Performance Metrics

Due to the subjective nature of optimal pathfinding, the navigation model's correctness was evaluated using qualitative human-in-the-loop metrics. For every frame, the system recorded the calculated risk metrics for each zone alongside the final navigational command. Reviewers then manually graded the generated command as either successful or unsuccessful based on the visual context. Additionally, quantitative runtime metrics were logged, capturing the precise latency (in milliseconds) of the navigation decision-making step, independent of the YOLO and depth estimation inference times, to guarantee real-time performance.

4.4. Thresholds and Navigational Logic

The system relies on a weighted risk-based and coverage-based heuristic model to classify the safety of the left, center, and right zones. Distance calculations act as the primary safety mechanism. A "far distance threshold" is set at 2.0 meters; objects detected beyond this distance accumulate zero risk score. Conversely, a "close distance threshold" applies a heavy risk multiplier of 2.0 to any objects detected within 1.5 meters. To prevent mathematical overflow when calculating inverse distance, the minimum distance floor is strictly clamped at 0.5 meters, with a maximum risk cap of 8.0 per object.

Alternatively, the system evaluates spatial density using an "unpassable coverage threshold." By calculating the total overlap area of detected bounding boxes against the total area of a specific zone, the system determines the zone is blocked if the obstacle coverage exceeds 40 percent.

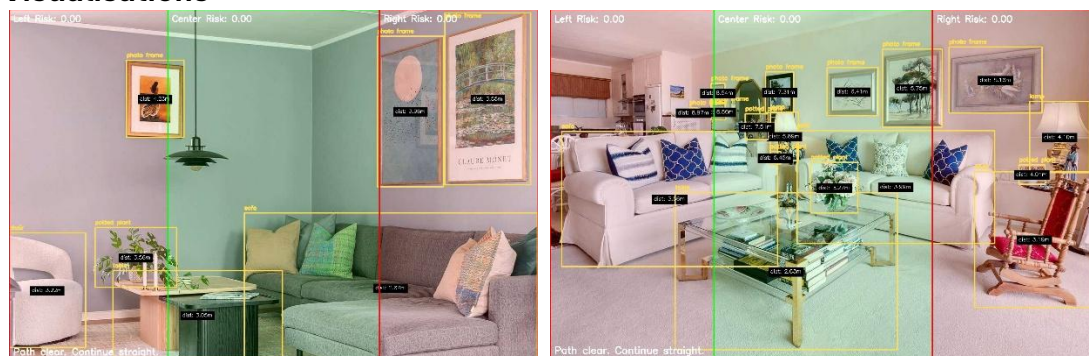
For temporal stability, the module employs an Exponential Moving Average with an alpha of 0.3 to smooth frame-to-frame risk spikes. Furthermore, a switch debounce system is implemented, requiring the logic to hold a new command for 3 consecutive frames before officially updating the user's audio prompt, significantly reducing erratic command flickering.

4.5. Quantitative and Qualitative Results

The navigation logic executes in under 5 milliseconds per frame, making it the least resource-intensive component of the entire pipeline. When evaluated against the static dataset, the module successfully generated the correct pathing command (e.g., turning away from an immediate centre obstacle toward a clear side zone) in the majority of cases.

Visually, the module accurately overlays shaded spatial boundaries (applied at 10 percent opacity) to highlight free versus obstructed zones, cleanly tagging the nearest obstacle in a blocked zone with its respective distance. Sample outputs successfully demonstrated the module advising "Centre is passable. Move slightly forward and carefully" when obstacles were small and on the periphery, while aggressively commanding a stop when overlap exceeded the 40 percent threshold.

4.6. Visualisations



4.7. Error Analysis and Observations

Analysis of failed navigation cases revealed that mistakes made by the navigation module were almost entirely inherited from upstream inaccuracies rather than internal logic flaws. For instance, false positive bounding boxes generated by the YOLO model occasionally caused the logic to interpret an empty space as a close-range threat, forcing an unnecessary evasive maneuver. Similarly, when the monocular depth model failed to properly scale the context (e.g., interpreting a small object on a table as a massive object on the floor), the navigation formula's distance-based division excessively spiked the risk score. Moving forward, improving the confidence threshold of the bounding boxes before they are passed into the navigation formula will be necessary to reduce these phantom avoidance edge cases.

6.